

Mirai Memory and Agent Loop Flow

Two end-to-end flows: how context is assembled into each model call, and how the agent loops through tools, approval pauses, and final answers.

Tags: context assembly · approval branch · loop-back paths

Flow 1: Context Assembly

Row A — From user input to prompt

Step	Title	Description	Meta
1	User Send	ChatView receives input, resets memory_update count, attachments, loading state.	ChatView.tsx
→			
2	Start Turn	Agent.processMessage() resets abort flag, tool cache, scratchpad, and nudge flags.	Agent
→			
3	Refresh Memory	Before each model call, loadMemoryForAgent() calls memoryAPI.getAll().	.mirai/memories
→			
4	Build Prompt	getSystemPrompt() combines user profile, Jira metadata, and formatted memory.	getSystemPrompt()

Row B — Memory into system prompt

Step	Title	Description	Meta
1	Memory Files	user.md has authority. learned.md and issue examples are lower-priority hints.	MemoryService
— loaded by →			
2	MemoryStore	formatMemoryForPrompt() creates the always-loaded memory block.	Priority encoded here
— merged into →			
3	System Prompt	Memory, Jira environment knowledge, and user context become the leading model instruction.	prompts.ts
— plus →			

Step	Title	Description	Meta
4	Model Messages	The model receives system prompt plus the recent conversation window.	tool_summary filtered

Row C — History, scratchpad, and AI call

Step	Title	Description	Meta
1	Chat History	Stored chatHistory reloads into <code>Agent.conversationHistory</code> when a chat opens.	storage
— windowed as →			
2	Recent Context	Last 40 model-visible messages, chronological, excluding UI-only summaries.	conversationHistory
— augmented by →			
3	Scratchpad	Per-turn notes survive context eviction and prevent repeat reads/searches.	sessionScratchpad
— sent to →			
4	AI Call	<code>callAI()</code> sends the assembled context to reasoning or main model.	<code>callAI()</code>

Write-back flow

1. **memory_update / delete** — Agent requests memory changes through tools. (*one update per turn*)
2. → **executeMemoryTool()** — ChatView validates and calls memoryAPI methods. (*renderer guard*)
3. → **MemoryService.save** — Markdown files are rewritten and cache updates. (*electron service*)
4. → **Next AI Call** — `loadMemory` refreshes, so the saved note enters future prompts. (*loop back*)

Flow 2: Agent Loop

Row A — Loop start to final answer

Step	Title	Description	Meta
1	runAgentLoop()	Iteration starts unless abortFlag is set or maxIterations is reached.	default cap: 10

Step	Title	Description	Meta
→			
2	callAI()	Refreshes memory, appends scratchpad, sends message window, streams response.	reasoning or main model
→			
3	Decision: Tool calls returned?	Yes: inspect tools · No: finalize	
— no →			
4	Final Answer	Action-first and think-only guards may nudge once; otherwise response is saved.	assistant message

Row B — Tool execution and next iteration

Step	Title	Description	Meta
1	Decision: Requires confirmation?	Yes: pause · No: execute	
— no →			
2	Execute Tool	Jira, file, memory, or scratchpad tool runs immediately.	<code>executeTool()</code>
→			
3	Record Result	Tool result enters history as <code>[tool_result: name]</code> ; summary goes to UI only.	tool_summary filtered
— loop →			
4	Next Iteration	The model sees the result and decides whether to call more tools or answer.	back to <code>callAI()</code>

Row C — Pending actions and resume

Step	Title	Description	Meta
1	PendingAction	Confirmed Jira writes and attachments pause the loop for user approval.	<code>onPendingAction()</code>
— approve →			

Step	Title	Description	Meta
2	approveAction()	Runs the deferred tool or batch, then pushes the tool result.	resume point
→			
3	Scratchpad / Cache	Writes invalidate cache; useful actions append scratchpad notes.	same turn memory
— resume →			
4	runAgentLoop()	The loop resumes after approval and lets the model continue.	not a new user turn

Loop exit paths

- Final assistant response when no tool call remains.
- Pause on `PendingAction` until approve or reject.
- Abort boundary via `abortFlag`.
- Iteration-limit message when max loop count is reached.

Context feedback loop

- Tool results become future model context.
- Scratchpad notes preserve what happened this turn.
- Memory updates persist beyond the current chat.
- UI summaries remain visible to the user but hidden from the model.

Source map

Area	Location
Memory files	<code>.mirai/memories/user.md</code> , <code>learned.md</code> , <code>issue-types/*.md</code>
Memory service	<code>electron/services/memory.ts</code>
Prompt assembly	<code>src/types/memory.ts</code> + <code>src/agent/prompts.ts</code>
Agent loop	<code>src/agent/index.ts</code>
Renderer wiring	<code>src/components/ChatView.tsx</code>
Tool definitions	<code>src/agent/tools.ts</code>

Source: Cursor canvas `mirai-memory-agent-loop.canvas.tsx` — this HTML is the print/export view.